

Introducción

El formato XML se ha introducido en todos los campos (incluidos la traducción) por varios motivos, pero podemos destacar unos pocos:

- Es un formato estándar y abierto, con lo que se pueden crear documentos para cualquier tipo de aplicación.
- Hay toda una serie de herramientas estándar que permiten tratar archivos XML de una manera rápida y fácil. En este sentido, Python también dispone de muchas librerías y módulos que permiten crear y leer archivos XML de una manera muy sencilla.

Si no conoces bien el formato XML te recomiendo seguir el [tutorial de XML de W3Schools](#).

En esta unidad veremos diferentes técnicas para tratar archivos XML e insistiremos en la importancia de utilizar alguna de las librerías disponibles. Utilizaremos diversas técnicas para leer y obtener información de archivos XML:

- **Funciones de cadenas.** Nos servirán en los casos en que las búsquedas sean sencillas y en las que el archivo XML guarde un formato muy homogéneo.
- **Expresiones regulares:** nos permitirán realizar operaciones sencillas sobre XMLs. En aplicaciones reales sólo se debe escoger esta opción cuando queramos obtener una información muy concreta de los archivos. Para tratamientos más complejos, es mejor usar las otras opciones que encontrarás en esta misma unidad.
- **El paquete `xmltodict`** que nos transforma un archivo XML en un diccionario de Python.
- **Uso de la librería `xml.etree`,** que nos proporciona muchas funciones para el tratamiento de archivos XML.

En buena parte esta unidad utilizaremos como archivo XML uno que representa una base de datos de CDs de música. Se puede encontrar en el zip de archivos de la unidad. Aprovechad para abrirlo con un editor de textos y observad cómo es. Este archivo lo presentamos en tres versiones:

- `catalog.xml`: Es la versión básica, si lo abres verás que presenta un formato muy homogéneo.
- `catalog-mod.xml`: Es el mismo archivo y contiene la misma información, pero ahora la información de title presenta un salto de línea adicional.
- `catalog-mod2.xml`: Es el mismo archivo XML, pero toda la información está en una sola línea.

Para cada programa que presente será interesante observar si la estrategia que hemos utilizado funciona correctamente para las tres versiones de este archivo.

También dedicaremos algunos apartados al tratamiento de algunos de los formatos XML estándar en la traducción, como TMX y TBX. A partir de todos estos ejemplos serás capaz de crear tus propios programas para tratar cualquier archivo XML

Podrás encontrar mucha más información sobre Python y XML en los siguientes enlaces:

- <https://docs.python.org/3/library/xml.html>
- <https://docs.python.org/3/library/xml.etree.elementtree.html>

De esta unidad dispones de los siguientes archivos:

- Esta misma unidad en PDF: [10-XML-spa.pdf](#)
- Los programas y archivos necesarios: [programas10-spa.zip](#)

10.1. Tratamiento de archivos XML con funciones de cadena

Los archivos XML se pueden tratar con funciones estándar de cadena, como en el programa-10-1.py.

```
import codecs
```

```
entrada=codecs.open("catalog.xml","r",encoding="utf-8")
salida=codecs.open("catalog.txt","w",encoding="utf-8")
artista=""
titulo=""
año=""
for linea in entrada:
    linea=linea.rstrip().lstrip()
    if linea.startswith("</cd>"):
        if not artista=="" and not titulo=="" and not año=="":
            cadena=artista+"\t"+titulo+"\t"+año
            print(cadena)
            salida.write(cadena+"\n")
    if linea.startswith("<title>") and linea.endswith("</title>"):
        titulo=linea[7:-8]
    elif linea.startswith("<artist>") and linea.endswith("</artist>"):
        artista=linea[8:-9]
    elif linea.startswith("<year>") and linea.endswith("</year>"):
        año=linea[6:-7]
```

Si nos fijamos, en este código veremos que únicamente se utilizan funciones de cadena estándar como `startswith()` y `endswith()`, por ejemplo. Si lo ejecutamos, veremos que funciona correctamente y que a la salida nos da la información de artista, título y año.

```
Bob Dylan Empire Burlesque 1985
Bonnie Tyler Hide your heart 1988
Dolly Parton Greatest Hits 1982
Gary Moore Still got the blues 1990
```

...

Esta estrategia puede ser válida, pero se basa demasiado en la disposición física de las marcas y la información en el documento. Prueba ahora si este programa funciona bien para `catalog-mod.txt` y `catalog-mod2.txt`.

10.2. Tratamiento de archivos XML mediante expresiones regulares

Para evitar el problema de la dependencia de la disposición de la información, podemos hacer uso de expresiones regulares, como en el siguiente programa (`programa-10-2.py`)

```
import codecs
import re
entrada=codecs.open("catalog.xml","r",encoding="utf-8")
salida=codecs.open("catalog.txt","w",encoding="utf-8")
artista=""
titulo=""
año=""
for linea in entrada:
    linea=linea.rstrip().lstrip()
    if linea.startswith("</cd>"):
        if not artista=="" and not titulo=="" and not año=="":
            cadena=artista+"\t"+titulo+"\t"+año
            print(cadena)
            salida.write(cadena+"\n")
    else:
        m_titulo = re.search('<title>(.*?)</title>', linea)
```

```
if m_title:
    title = m_title.group(1)
m_artist = re.search('<artist>(.*?)</artist>', linia)
if m_artist:
    artist = m_artist.group(1)
m_year = re.search('<year>(.*?)</year>', linia)
if m_year:
    year = m_year.group(1)
```

Intenta ejecutar el programa con los archivos catalog modificados. ¿Los puede tratar todos?

10.3. Tratamiento de archivos XML con xmldict

xmldict nos permite tratar archivos XML de una manera muy fácil, ya que convierte los archivos XML en una estructura de datos de tipo diccionario. En el programa-10-3.py podemos observar cómo utilizar esta librería.

```
import xmldict

xml=open('catalog.xml')
xmldict = xmldict.parse(xml.read())

for cd in xmldict["catalog"]["cd"]:
    print(cd["artist"],cd["title"],cd["year"])
```

Prueba con todas las modificaciones del archivo catalog y observa si es capaz de procesar correctamente todos los archivos. Recuerda que si te aparece un mensaje como el siguiente:

```
Traceback (most recent call last):
File "programa-10-3.py", line 1, in <module>
import xmldict
ModuleNotFoundError: No module named 'xmldict'
```

quiere decir que no tienes instalado el módulo xmldict en tu ordenador. Para instalarlo puedes utilizar pip o pip3, haciendo

```
pip install xmldict
o
pip3 install xmldict
```

o dependiendo del sistema operativo y los permisos

```
sudo pip3 install xmldict
```

10.4. Tratamiento de archivos XML con xml.etree

Disponemos de una serie de librerías que nos facilitan mucho la lectura de archivos XML. Una de ellas es [xml.etree.ElementTree](#). Para observar cómo funciona, ejecutaremos el programa-10-4.py y observaremos la salida:

```
import xml.etree.ElementTree as etree
```

```
for event, elem in etree.iterparse("catalog.xml", events=("start", "end")):
    print(event, elem, elem.tag, elem.attrib)
```

La librería es capaz de detectar cuando hay un evento (y hemos seleccionado el principio (start) y el final (end)), el elemento afectado, la etiqueta del elemento y el atributo del elemento. En este programa de prueba simplemente escribimos esta información:

```
start <Element 'catalog' at 0x7f456b5c3728> catalog {}
start <Element 'cd' at 0x7f4569d999f8> cd {'id': '1'}
start <Element 'title' at 0x7f4569d3c2c8> title {}
end <Element 'title' at 0x7f4569d3c2c8> title {}
start <Element 'artist' at 0x7f4569d3c318> artist {}
end <Element 'artist' at 0x7f4569d3c318> artist {}
```

Con esta información, podemos hacer un programa que lea el archivo (programa-10.5.py):

```
import xml.etree.ElementTree as etree
import codecs
artist=""
title=""
year=""
sortida=codecs.open("catalog.txt", "w", encoding="utf-8")
for event, elem in etree.iterparse("catalog.xml", events=("start", "end")):
    if event=="end" and elem.tag=="cd":
        cadena=artist+"\t"+title+"\t"+year
        print(cadena)
        sortida.write(cadena+"\n")
        artist=""
        title=""
        year=""
    if event=="end" and elem.tag=="title":
        title="".join(elem.itertext()).lstrip().rstrip()
    if event=="end" and elem.tag=="artist":
        artist="".join(elem.itertext()).lstrip().rstrip()
    if event=="end" and elem.tag=="year":
        year="".join(elem.itertext()).lstrip().rstrip()
```

Si te fijas bien en el programa verás que iteramos sobre todos los eventos que suceden al leer el archivo XML (con la instrucción for) y que después mediante instrucciones if actuamos dependiendo de qué etiqueta (elem.tag) se ha cerrado (event=="end")

Comprueba que este programa funciona también bien con el catalog modificado.

10.5. Tratamiento de archivos TMX

En este apartado explicamos el desarrollo de un programa que transforma archivos TMX en texto separado por tabuladores. En esta unidad hemos visto diversas estrategias para el tratamiento de archivos XML. Habitualmente utilizaremos alguna librería especializada en el tratamiento de archivos XML, como por ejemplo la librería [xml.etree.ElementTree](#), que ya conocemos de esta unidad.

En este apartado trabajaremos con un archivo TMX de ejemplo: en-es.tmx (que se corresponde al corpus ECB (*European Central Bank*) inglés-español).

10.5.a. TMX

Como ya sabemos, el formato TMX (*Translation Memory eXchange*) es un formato estándar basado en XML para el intercambio de memorias de traducción. A continuación podemos observar un fragmento de archivo TMX:

```
<body>
  <tu>
    <tuv xml:lang="en"><seg>The European Central Bank</seg></tuv>
    <tuv xml:lang="es"><seg>Banco Central Europeo</seg></tuv>
  </tu>
  ...
```

Queremos desarrollar un programa que a partir de un archivo TMX, el código de lengua 1 y el código de lengua 2, cree un archivo separado por tabuladores que contenga segmento en la lengua 1, un tabulador y el segmento en la lengua 2. Nuestro programa tendrá que leer el archivo XML y recorrer los elementos `tu`:

```
tree = ET.parse(fentrada)
root = tree.getroot()
for tu in root.iter('tu'):

    De cada tu tendrá que recorrer todos los tuv:

    for tuv in tu.iter('tuv'):

        mirar el código de lengua:

        lang=tuv.attrib['{http://www.w3.org/XML/1998/namespace}lang']

        coger el texto del seg:

        text=seg.text
```

y mirar si corresponde a la lengua 1 o a la lengua 2 para asignar el texto leído al correspondiente segmento de la lengua 1 o de la lengua 2:

```
if lang==l1: sl_text=text
elif lang==l2: tl_text=text
```

y por último escribir en el archivo (y también mostrar por pantalla) los textos correspondientes a la lengua 1 y a la lengua 2, comprobando antes que exista texto para las dos lenguas:

```
if not sl_text=="" and not tl_text=="":
    cadena=sl_text+"\t"+tl_text
    print(cadena)
    sortida.write(cadena+"\n")
```

Fíjate que cada vez que leemos una `tu` ponemos a "" (forzamos que sean cadenas vacías) los textos de la lengua 1 y la lengua 2.

10.5.b. Código completo del programa

El programa completo lo encontramos en TMX2tabtxt.py:

```
import xml.etree.ElementTree as ET
import sys
import codecs

fentrada=sys.argv[1]
fsalida=sys.argv[2]
l1=sys.argv[3]
l2=sys.argv[4]

minrel=3
nmax=5
salida=codecs.open(fsalida,"w",encoding="utf-8")

tree = ET.parse(fentrada)
root = tree.getroot()

for tu in root.iter('tu'):
    sl_text=""
    tl_text=""
    for tuv in tu.iter('tuv'):
        lang=tuv.attrib['{http://www.w3.org/XML/1998/namespace}lang']
        for seg in tuv.iter('seg'):
            text=seg.text
            if lang==l1: sl_text=text
            elif lang==l2: tl_text=text

    if not sl_text=="" and not tl_text=="":
        cadena=sl_text+"\t"+tl_text
        print(cadena)
        salida.write(cadena+"\n")
```

Fíjate que el programa espera que demos como parámetros de entrada el archivo TMX a transformar y los códigos de lengua:

```
fentrada=sys.argv[1]
fsortida=sys.argv[2]
l1=sys.argv[3]
l2=sys.argv[4]
```

para ejecutar el programa tendremos que escribir:

```
python3 TMX2tabtxt.py en-es.tmx memo-en-es.txt en es
```

y en el archivo memo-en-es.txt tendremos la memoria de traducción convertida a texto tabulado.

Mediante la librería sys podemos obtener los parámetros de entrada del programa de manera muy sencilla. En la lista sys.argv tenemos los parámetros, pero es muy importante tener en cuenta que en sys.argv[0] tenemos el nombre del programa y que el primer parámetro está en sys.argv[1].

10.5.b. Escritura de archivosTMX

En el apartado anterior hemos visto como leer un archivo TMX y crear un archivo de texto tabulado. Ahora aprenderemos a escribir archivos TMX haciendo un programa que pase de un archivo de texto tabulado a un archivo TMX. Presentaremos dos versiones del programa. La primera versión simplemente utilizará funciones de cadena para escribirlas en el archivo resultante. El programa es tabtxt2TMXa.py:

```
import sys
import codecs
from xml.sax.saxutils import escape

fentrada=sys.argv[1]
fsalida=sys.argv[2]
l1=sys.argv[3]
l2=sys.argv[4]

entrada=codecs.open(fentrada,"r",encoding="utf-8")
salida=codecs.open(fsalida,"w",encoding="utf-8")

cadena='<?xml version="1.0" encoding="UTF-8" ?>'
salida.write(cadena+"\n")
cadena='<tmx version="1.4">'
salida.write(cadena+"\n")
cadena='<header/>'
salida.write(cadena+"\n")
cadena=' <body>'
salida.write(cadena+"\n")

for linea in entrada:
    linea=linea.rstrip()
    camps=linea.split("\t")
    segment1=camps[0]
    segment2=camps[1]
    cadena=' <tu>'
    salida.write(cadena+"\n")
    cadena=' <tuv xml:lang="'+l1+'"><seg>'+escape(segment1)+'</seg></tuv>'
    salida.write(cadena+"\n")
    cadena=' <tuv xml:lang="'+l2+'"><seg>'+escape(segment2)+'</seg></tuv>'
    salida.write(cadena+"\n")
    cadena=' </tu>'
    salida.write(cadena+"\n")

cadena=' </body>'
salida.write(cadena+"\n")
cadena='</tmx>'
salida.write(cadena+"\n")
```

Como puedes ver, vamos creando las cadena necesarias concatenando los elementos y los escribimos en el archivo con write. Esta es una opción muy sencilla y eficiente. Para utilizar este programa tenemos que escribir:

```
python3 tabtxt2TMXa.py archivo.txt archivo.tmx en es
```

(cambiando archivo.txt por el nombre del archivo de texto tabulado a convertir y archivo.tmx por el nombre del archivo TMX que queremos crear y utilizando los códigos de lengua adecuados).

También es posible utilizar una librería específica para escribir archivos XML, como el programa tabtxt2TMXb.py. Como veremos, simplemente va construyendo los elementos con sus elementos hijo, etc. y después lo escribe todo en disco. La función prettify hace que la salida tenga un mejor aspecto visual.

```
import sys
import codecs
```

```
from xml.etree.ElementTree import Element, SubElement, Comment, tostring
from xml.etree import ElementTree
from xml.dom import minidom
```

```
def prettify(elem):
    """Return a pretty-printed XML string for the Element.
    """
    rough_string = ElementTree.tostring(elem, 'utf-8')
    reparsed = minidom.parseString(rough_string)
    return reparsed.toprettyxml(indent="  ")

fentrada=sys.argv[1]
fsalida=sys.argv[2]
l1=sys.argv[3]
l2=sys.argv[4]

entrada=codecs.open(fentrada,"r",encoding="utf-8")
salida=codecs.open(fsalida,"w",encoding="utf-8")

# Configure one attribute with set()
root = Element('tmx')
root.set('version', '1.4')

header = SubElement(root, 'header')

body = SubElement(root, 'body')

for linea in entrada:
    linea=linea.rstrip()
    camps=linea.split("\t")
    segment1=camps[0]
    segment2=camps[1]
    tu=SubElement(body, 'tu')

    tuv = SubElement(tu,'tuv')
    tuv.set('xml:lang',l1)
    seg= SubElement(tuv,'seg')
    seg.text=segment1

    tuv = SubElement(tu,'tuv')
    tuv.set('xml:lang',l2)
    seg= SubElement(tuv,'seg')
    seg.text=segment2
salida.write((prettify(root)))
```

El inconveniente de esta estrategia es que vamos ocupando memoria del sistema hasta que guardamos el archivo en el disco y esto para archivos muy grandes puede dar problemas de memoria. La ventaja es que utilizando esta librería nos aseguramos que el archivo de salida estará bien formado.

Este programa se utiliza igual que el anterior:

```
python3 tabtxt2TMXb.py archivo.txt archivo.tmx en es
```

10.6. Tratamiento de archivos TBX

En este apartado vamos a aprender a tratar archivos TBX. Lo haremos de una manera muy similar a los archivos TMX. Para poder tratarlos necesitamos conocer un poco cómo son por dentro. Tenéis disponible el archivo ejemplo.tbx (mostramos solo un fragmento):

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE martif SYSTEM "TBXcoreStructV02.dtd">
<martif type="TBX-Default" xml:lang="en">
  <martifHeader>
    <fileDesc>
      <sourceDesc>
        <p>This is a TBX file downloaded from the IATE website. Address any enquiries to iate@cdt.europa.eu.</p>
      </sourceDesc>
    </fileDesc>
    <encodingDesc>
      <p type="XCSURI">TBXXCS.xcs</p>
    </encodingDesc>
  </martifHeader>
  <text>
    <body>
      <termEntry id="IATE-28960">
        <descripGrp>
          <descrip type="subjectField">2826, 7611</descrip>
        </descripGrp>
        <langSet xml:lang="en">
          <tig>
            <term>EBU</term>
            <termNote type="termType">abbreviation</termNote>
            <descrip type="reliabilityCode">3</descrip>
          </tig>
          <tig>
            <term>European Blind Union</term>
            <termNote type="termType">fullForm</termNote>
            <descrip type="reliabilityCode">3</descrip>
          </tig>
        </langSet>
        <langSet xml:lang="es">
          <tig>
```

```
<term>UEC</term>
<termNote type="termType">abbreviation</termNote>
<descrip type="reliabilityCode">3</descrip>
</tig>
<tig>
  <term>Unión Europea de Ciegos</term>
  <termNote type="termType">fullForm</termNote>
  <descrip type="reliabilityCode">3</descrip>
</tig>
</langSet>
<langSet xml:lang="fr">
  <tig>
    <term>UEA</term>
    <termNote type="termType">abbreviation</termNote>
    <descrip type="reliabilityCode">2</descrip>
  </tig>
  <tig>
    <term>Union européenne des aveugles</term>
    <termNote type="termType">fullForm</termNote>
    <descrip type="reliabilityCode">3</descrip>
  </tig>
</langSet>
</termEntry>
....
```

En el primer programa (TBX2tabtxt.py) vamos a convertir el TBX en texto tabulado. Fíjate en cómo pedimos los parámetros de entrada:

```
import xml.etree.ElementTree as ET
import sys
import codecs

fentrada=sys.argv[1]
fsalida=sys.argv[2]
l1=sys.argv[3]
l2=sys.argv[4]

minrel=3
nmax=5
salida=codecs.open(fsalida,"w",encoding="utf-8")

tree = ET.parse(fentrada)
root = tree.getroot()

for termEntry in root.iter('termEntry'):
```

```
termL1=""
termL2=""
for langset in termEntry.iter('langSet'):
    lang=langset.attrib['{http://www.w3.org/XML/1998/namespace}lang']
    for tig in langset.iter('tig'):
        for term in tig.iter('term'):
            termino=term.text
            if lang==l1:
                termL1=termino
            elif lang==l2:
                termL2=termino
    if not termL1=="" and not termL2=="":
        cadena=termL1+"\t"+termL2
        print(cadena)
        salida.write(cadena+"\n")
```

Para ejecutar el programa tendremos que escribir:

python3 TBX2tabtxt.py ejemplo.tbx ejemplo.txt en es

Hemos introducido una modificación en el programa TBX2tabtxt2.py para que en la salida nos proporcione el ID y las áreas temáticas:

```
import xml.etree.ElementTree as ET
import sys
import codecs

fentrada=sys.argv[1]
fsalida=sys.argv[2]
l1=sys.argv[3]
l2=sys.argv[4]

minrel=3
nmax=5
salida=codecs.open(fsalida,"w",encoding="utf-8")

tree = ET.parse(fentrada)
root = tree.getroot()

for termEntry in root.iter('termEntry'):
    termL1=""
    termL2=""
    ident=termEntry.attrib['id']
    subjects=""
    for group in termEntry.iter('descripGrp'):

        for descrip in group.iter('descrip'):
            if descrip.attrib['type']=='subjectField':
                subjects=descrip.text
    for langset in termEntry.iter('langSet'):
        lang=langset.attrib['{http://www.w3.org/XML/1998/namespace}lang']
        for tig in langset.iter('tig'):
            for term in tig.iter('term'):
                termino=term.text
                if lang==l1:
                    termL1=termino
                elif lang==l2:
                    termL2=termino
    if not termL1=="" and not termL2=="":
        cadena=ident+"\t"+subjects+"\t"+termL1+"\t"+termL2
```

```
print(cadena)
salida.write(cadena+"\n")
```

Y una modificación más (TBX2tabtxt3.py) para que solo se escriba el término si el reliabilityCode es 3 o superior:

```
import xml.etree.ElementTree as ET
import sys
import codecs

fentrada=sys.argv[1]
fsalida=sys.argv[2]
l1=sys.argv[3]
l2=sys.argv[4]

minrel=3
nmax=5
salida=codecs.open(fsalida,"w",encoding="utf-8")

tree = ET.parse(fentrada)
root = tree.getroot()

for termEntry in root.iter('termEntry'):
    termL1=""
    termL2=""
    ident=termEntry.attrib['id']
    subjects=""
    reliability=0
    for group in termEntry.iter('descripGrp'):

        for descrip in group.iter('descrip'):
            if descrip.attrib['type']=='subjectField':
                subjects=descrip.text
        for langset in termEntry.iter('langSet'):
            lang=langset.attrib['{http://www.w3.org/XML/1998/namespace}lang']
            for tig in langset.iter('tig'):
                for term in tig.iter('term'):
                    termino=term.text
                    if lang==l1:
                        termL1=termino
                    elif lang==l2:
                        termL2=termino
                for descrip in tig.iter('descrip'):
                    if descrip.attrib['type']=='reliabilityCode':
                        reliability=int(descrip.text)

    if not termL1=="" and not termL2=="" and reliability>=3:
        cadena=ident+"\t"+subjects+"\t"+termL1+"\t"+termL2
        print(cadena)
        salida.write(cadena+"\n")
```

Ejercicio: Escribe un programa que transforme un archivo de texto tabulado en un archivo TBX. Puedes utilizar como entrada la salida del programa TBX2tabtxt3.py. Para solucionar este ejercicio puedes utilizar como ejemplo los programas del apartado 10.5.b Escritura de archivos TBX.

10.7. Caso práctico: Terminología Oberta del TermCat

El TermCat es el centro de terminología de la Generalitat de Catalunya. Este centro publica numerosos glosarios terminológicos en los que se encuentran los términos en catalán, español e inglés (y en algunos casos en más lenguas). Buena parte de estos glosarios se publican para uso libre en la colección [Terminologia Oberta](#). Los glosarios de esta colección se publican en un formato XML que, por desgracia, no es estándar.

Con lo que hemos aprendido en esta unidad podremos crear programas que transformen los glosarios de terminología oberta en archivos TBX o en archivos de texto tabulado.

Descarga cualquier glosario de Terminologia Oberta (o utiliza el archivo cdlimmunologia.xml que proporcionamos con los archivos de esta unidad).

Intenta desarrollar un programa para transformar un archivo XML de Terminologia Oberta a TMX y otro para transformarlo a texto tabulado. Tienes estos programas desarrollados en <https://github.com/aoliverg/TO2TBX> pero inténtalo tu antes de consultarlos.